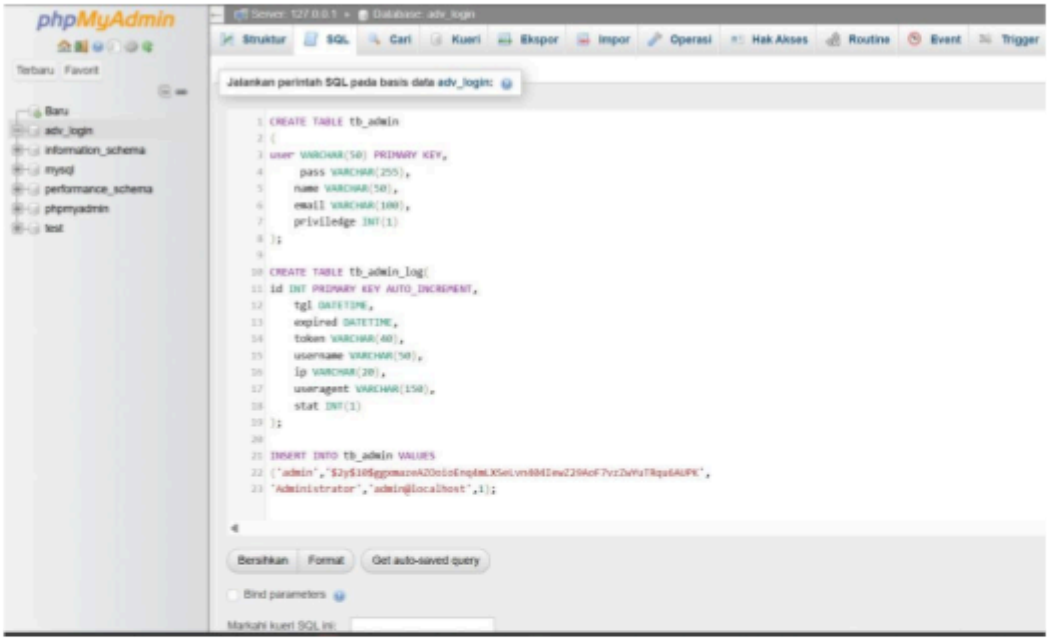
LEMBAR KERJA PRAKTIKUM

Silakan lakukan praktikum sesuai instruksi di atas, dan dokumentasikan pada lembar kerja praktikum berikut:

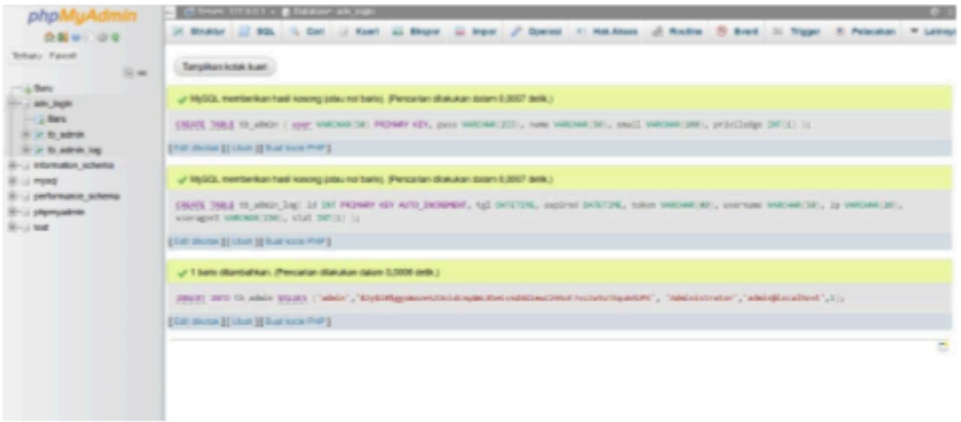
Nama	**: Faris Daffa Al Haq**	**Praktikum**	**: 9**
NIM	**: J0403251042**	**Ke-Tanggal**	**: 22 Oktober 2026**

Pembuatan Database dan Struktur Tabel

Buka phpMyAdmin dan buat database baru bernama adv_login.



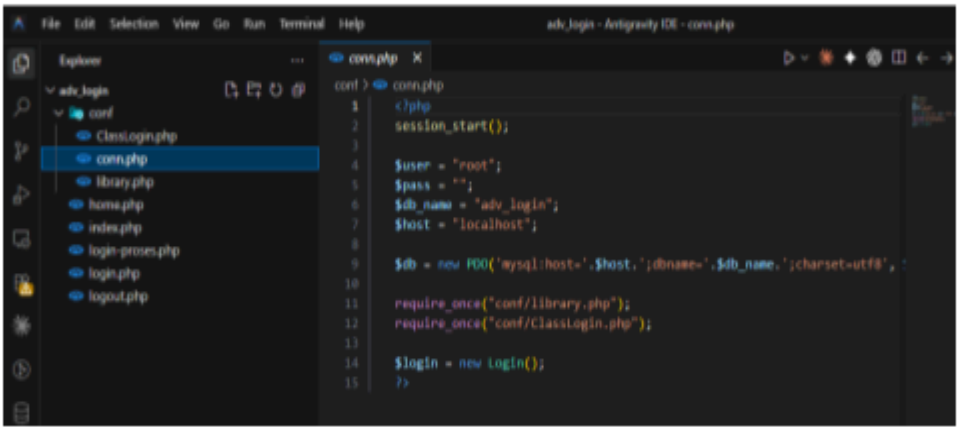
Jalankan SQL ini di database yang sudah dibuat.



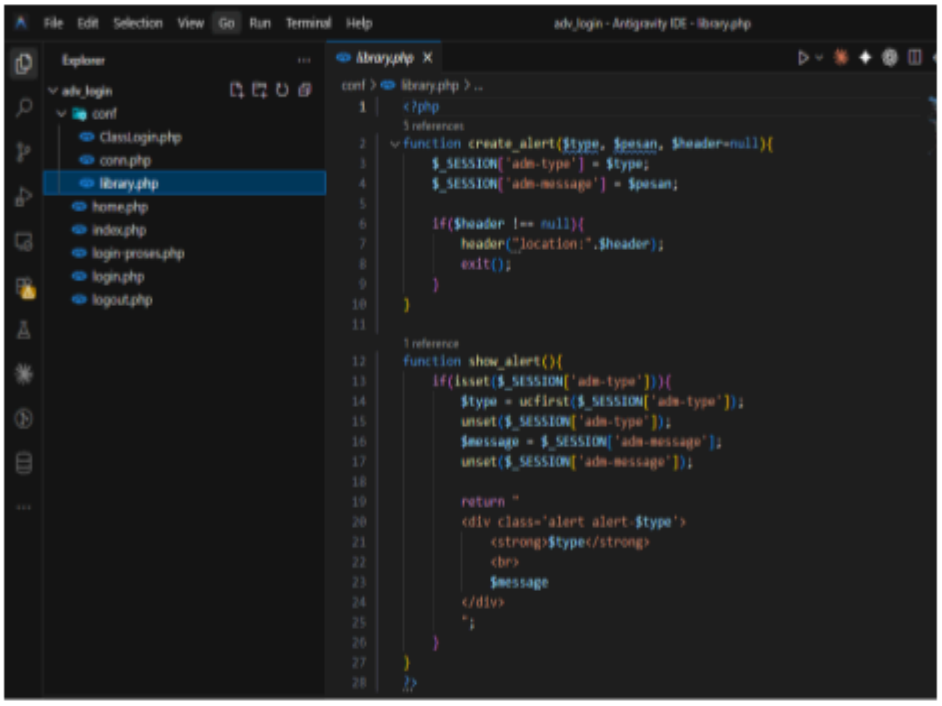
2. Koneksi dan File Library

a) Di folder project, buat sebuah folder conf untuk menampung file penting kita yaitu conn.php, library.php, dan ClassLogin.php.

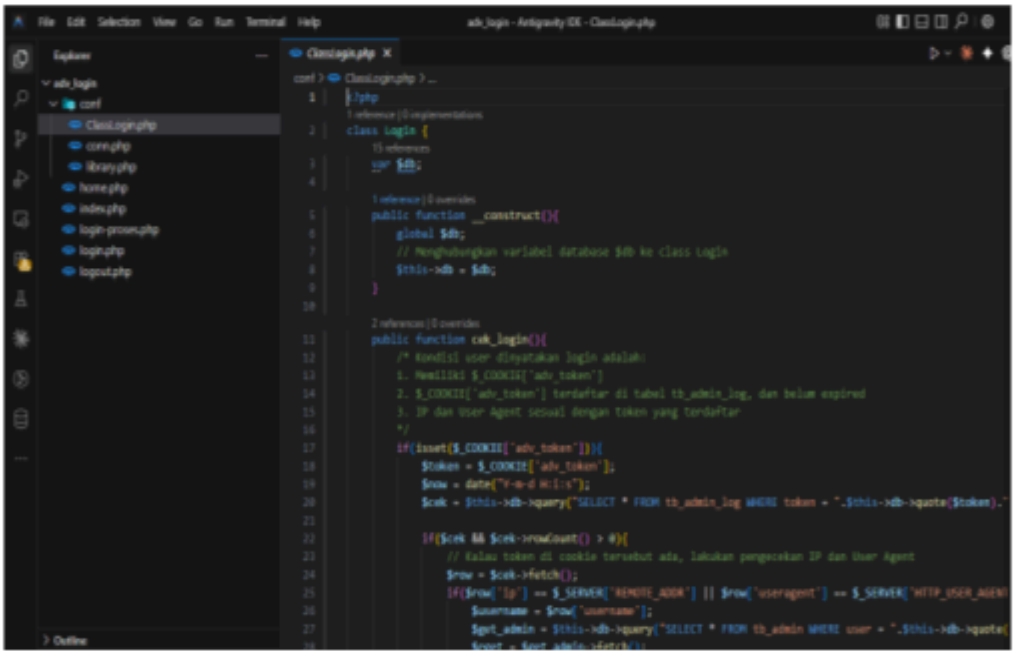
(1) File conn.php

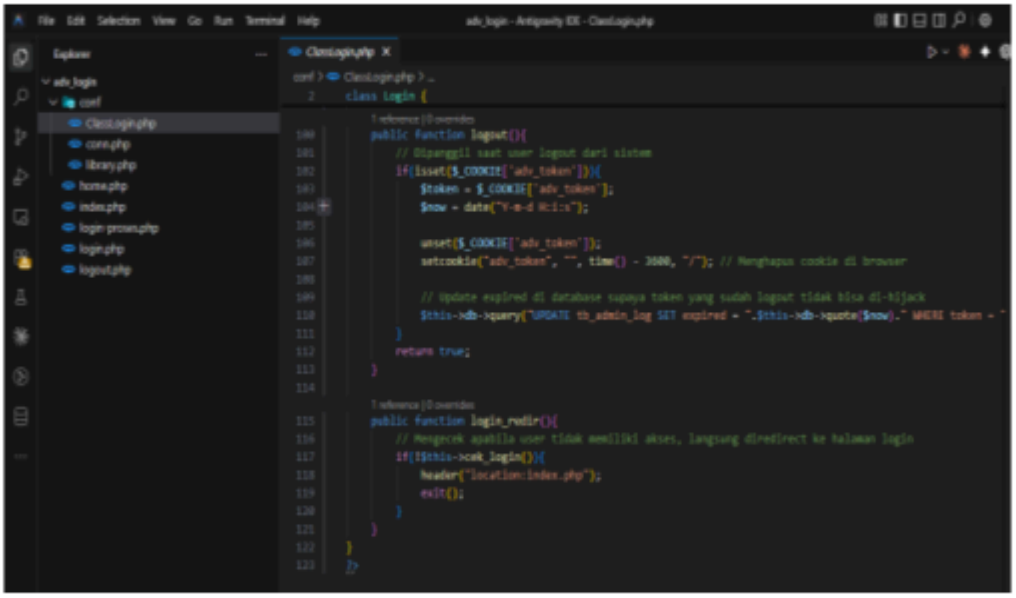


(2) File library.php

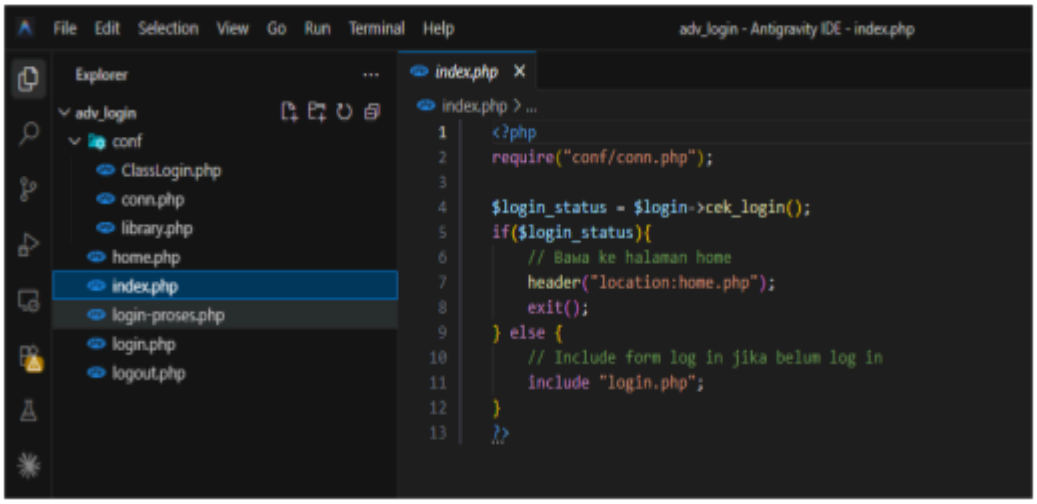


(3) File ClassLogin.php

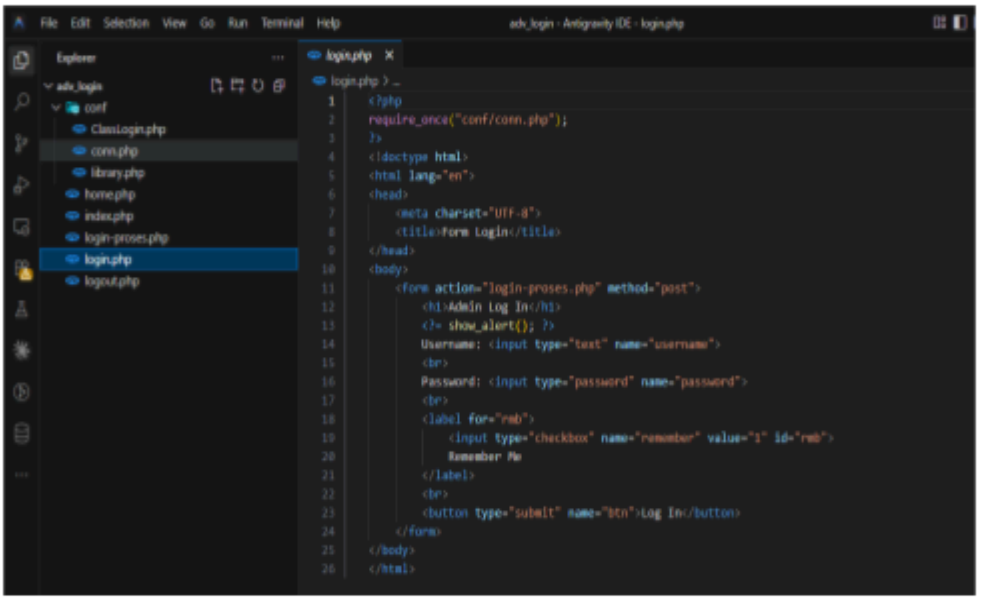




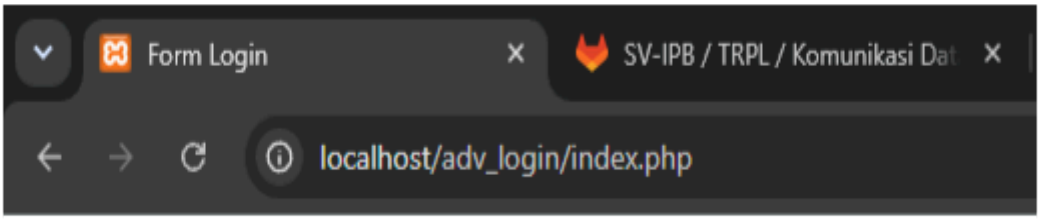
b) Buat file index.php di direktori utama project.



c) Buat form login di file login.php.



d) Buka di browser, hasil decode akan terlihat seperti berikut:

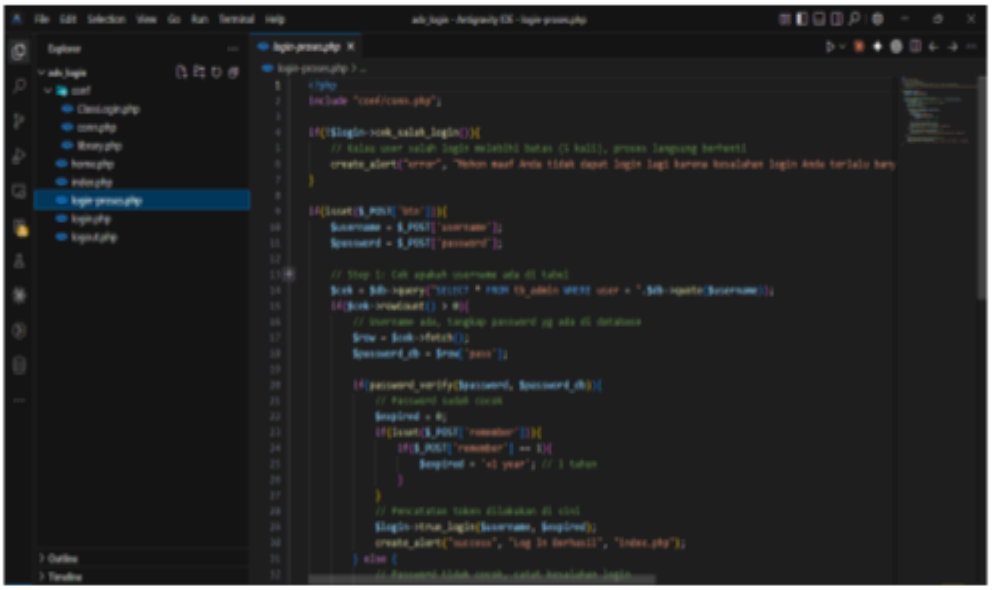


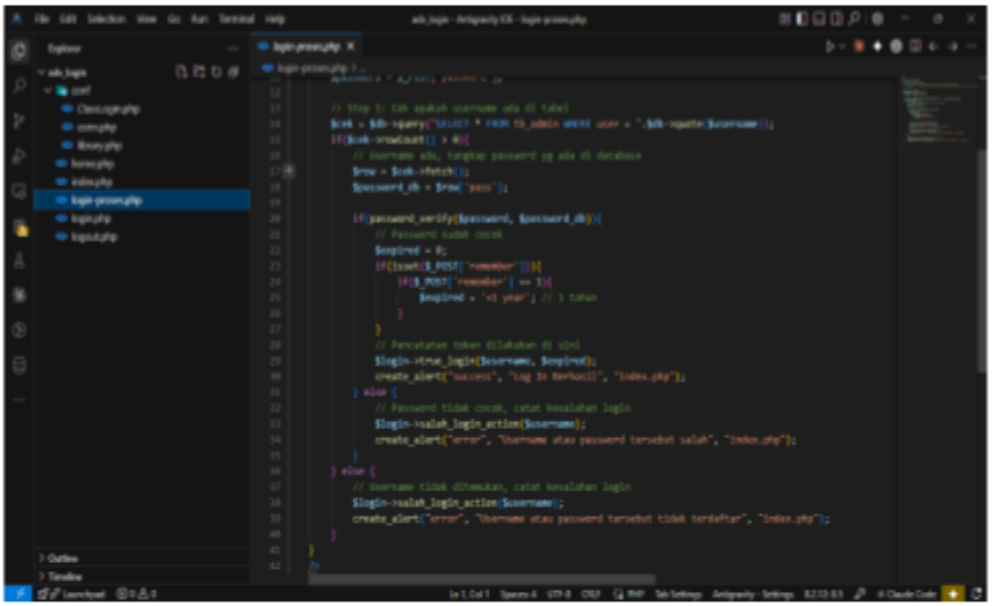
Admin Log In

Success
Anda sudah logout dari sistem
Username:
Password:
Remember Me
Log In

3. Proses Login

a) Membuat file login-proses.php untuk validasi login.

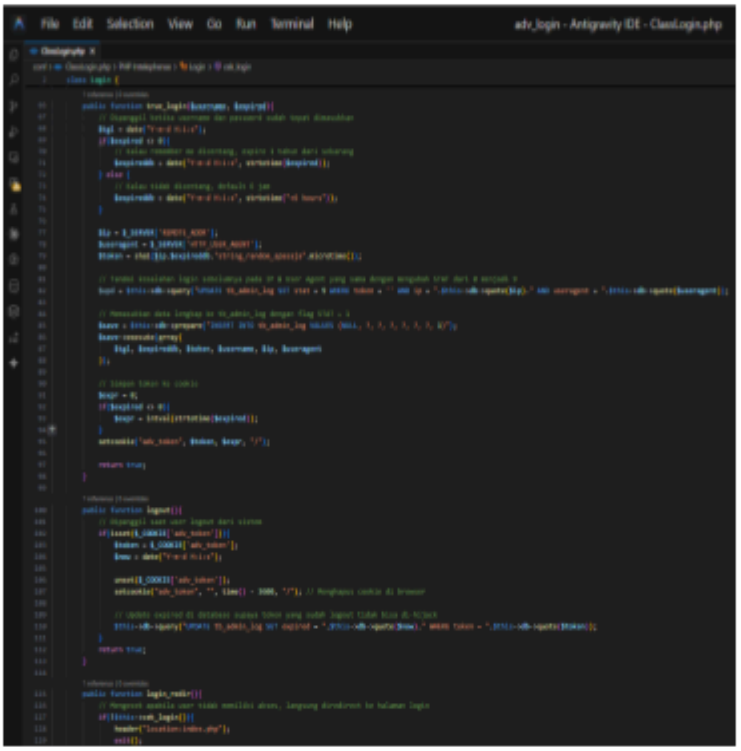




4. Penyesuaian Class Login

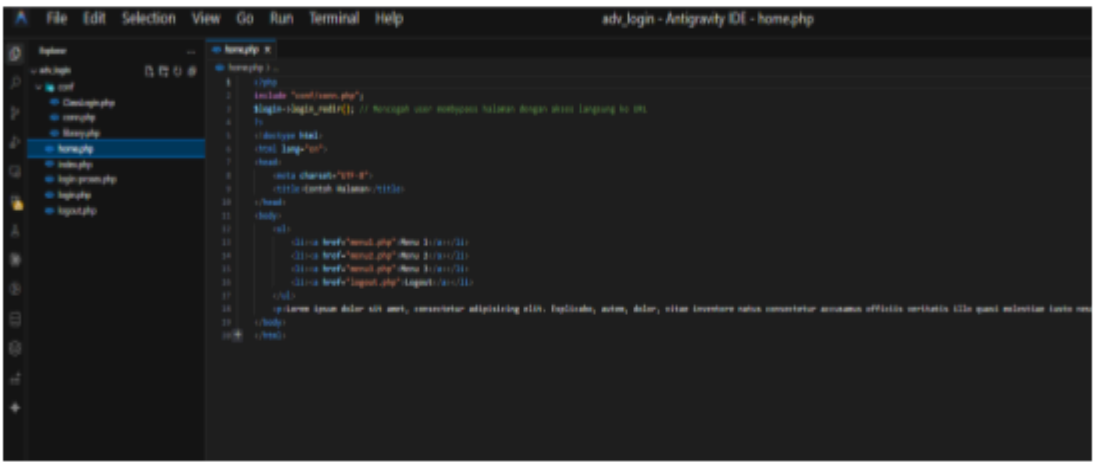
a) Salin method-method dibawah ini, letakkan sesuai di class Login yang sudah ada tadi.



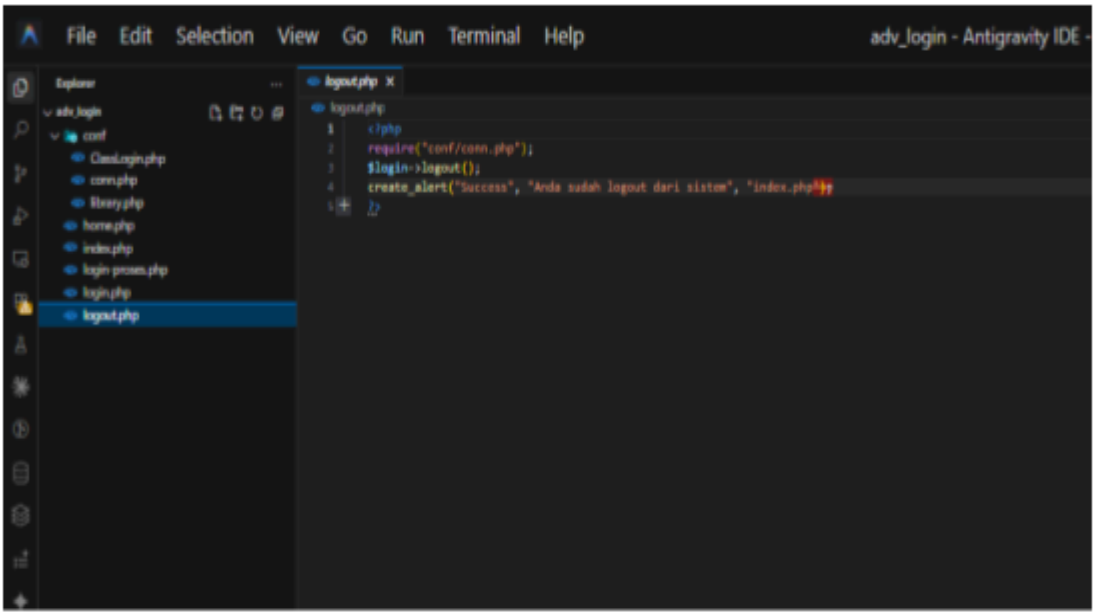


5. Pengujian

a) Buat file tambahan di direktori utama : home.php.



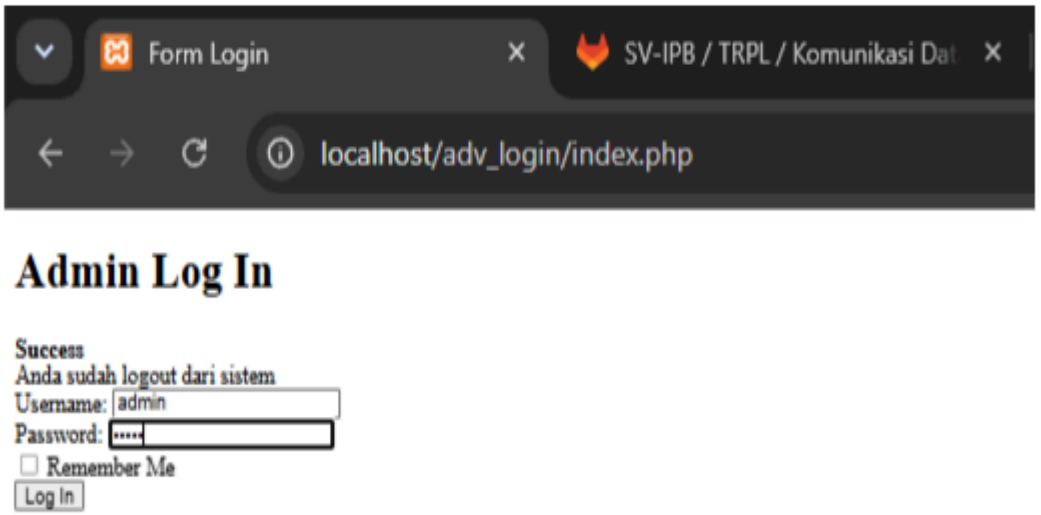
b) Buat file logout di direktori utama : logout.php.

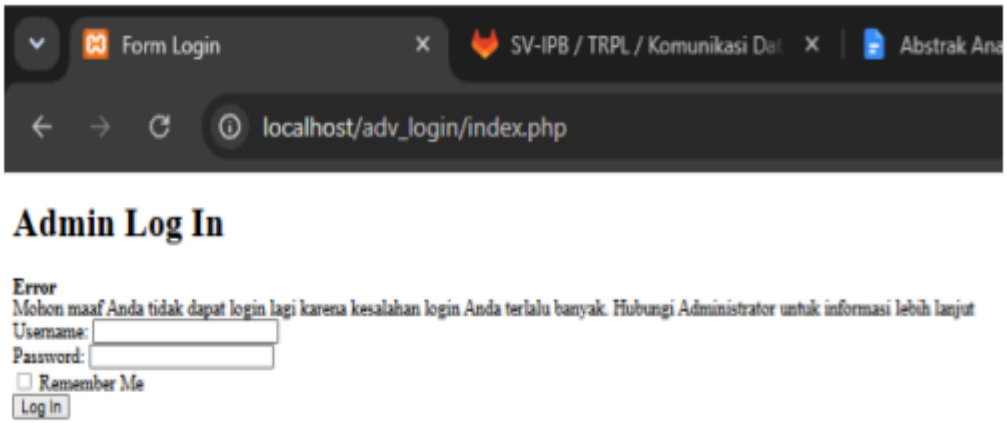


6. Pembuktian Hasil

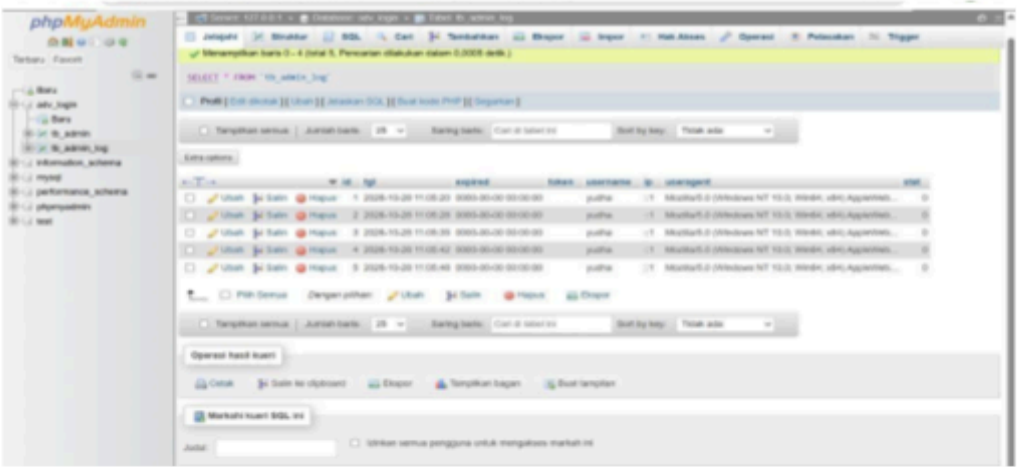
a) Pertama, metode Bruteforce tidak akan berjalan di halaman Log In ini, karena sudah ada fungsi cek_salah_login() yang akan menutup akses login ketika sudah mencapai sekian kali kesalahan.

Contoh login menggunakan username/password yang salah : daffa





Setelah lima kali memasukkan password yang salah, sistem menolak percobaan berikutnya dengan pesan bahwa login tidak dapat dilakukan lagi. Hal ini terjadi karena method cek_salah_login() menghitung jumlah catatan stat = 0 dari IP yang sama di tabel tb_admin_log. Ketika jumlahnya mencapai batas (5), proses login dihentikan sebelum password diperiksa. Penolakan tetap terjadi walau password yang dimasukkan benar, sehingga penyerang tidak bisa terus menebak password.

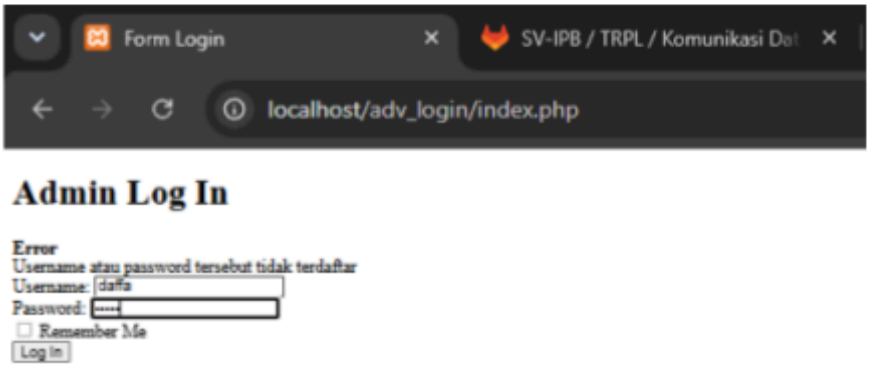


Setiap login gagal dicatat melalui salah_login_action() beserta waktu, username, IP, dan User-Agent dengan stat = 0. Lima baris dengan IP yang sama inilah yang memicu pemblokiran akses masuk.

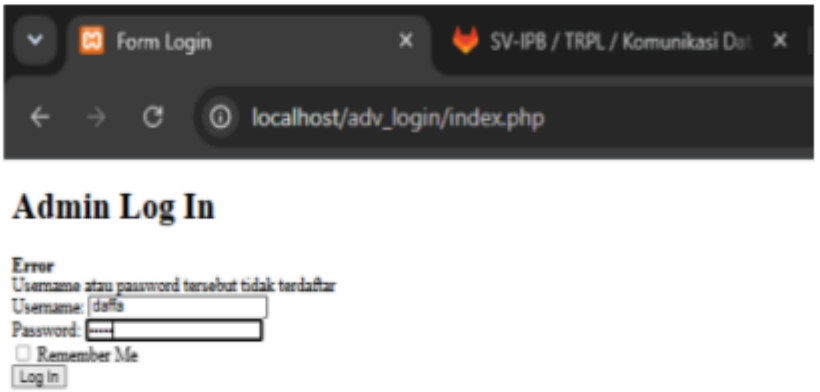
b) Kedua, metode SQL Injection juga nggak akan berjalan, karena menggunakan escaping ($db->quote), dan fungsi prepared statement di semua query yang ada.

(1) Di form login, coba satu per satu login dengan username dibawah ini (password bebas):

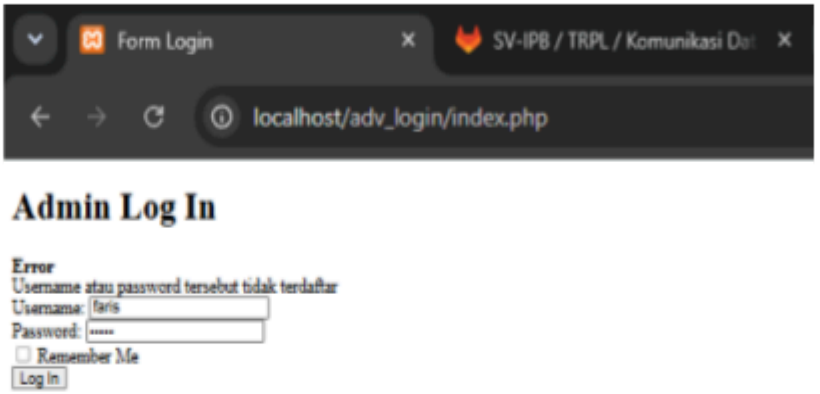
- daffa



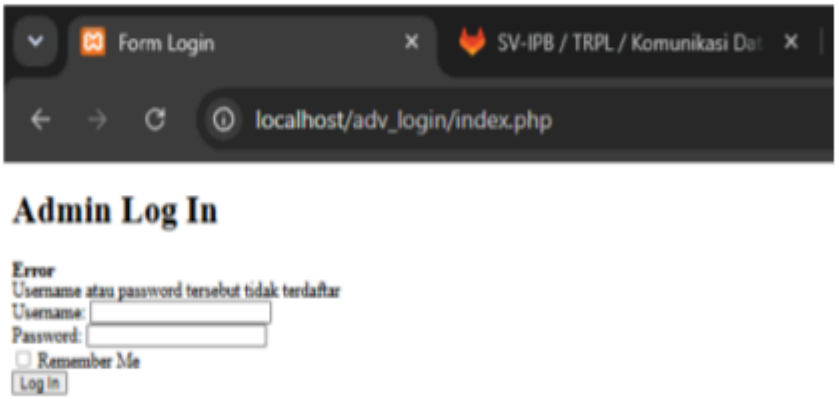
Output:



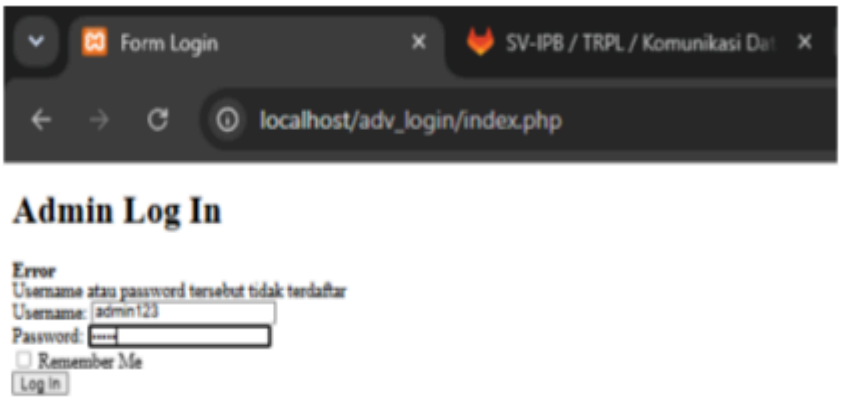
- faris



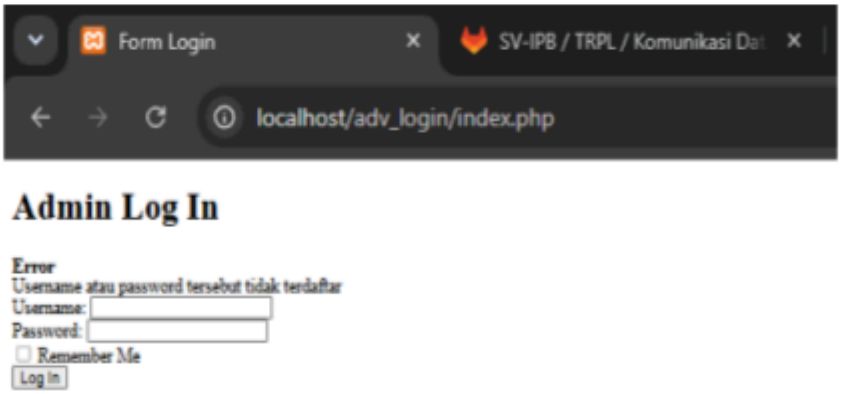
Output:

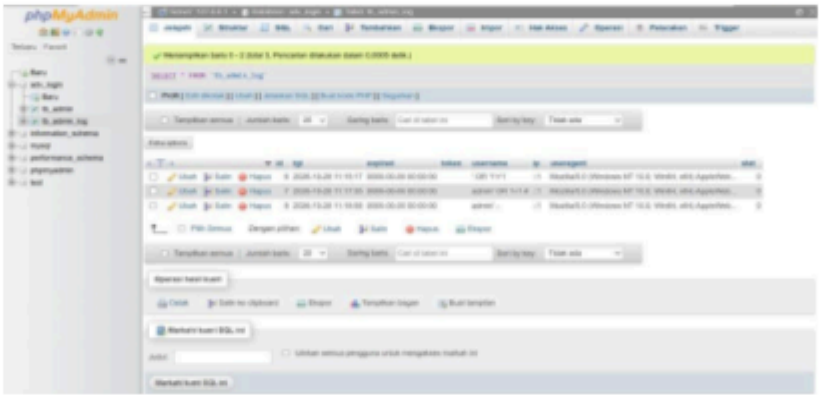


- admin123



Output:





Dari ketiga Input injeksi SQL di atas, tidak ada yang berhasil melewati login dan sistem menampilkan pesan "Username atau password tersebut tidak terdaftar". Hal ini terjadi karena input diproses dengan $db->quote(), yang meng-escape tanda kutip sehingga seluruh input dianggap sebagai satu string data, bukan bagian dari perintah SQL.

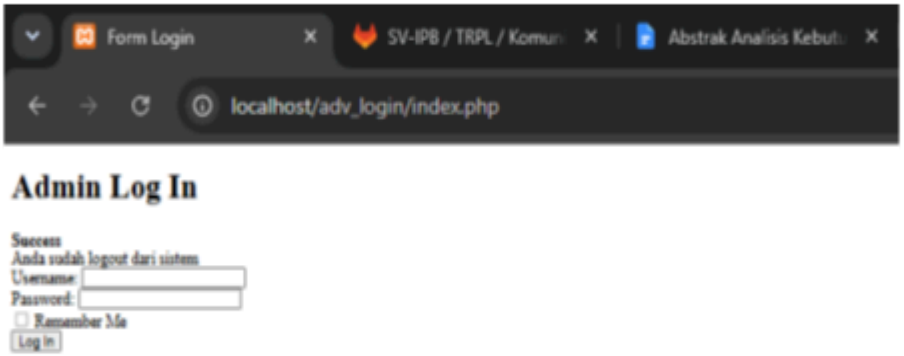
tb_admin_log mencatat input injeksi sebagai teks biasa Pada kolom username tersimpan teks daffa, faris dan admin123 apa adanya. Ini membuktikan bahwa input tersebut diperlakukan sebagai data dan tidak pernah dieksekusi sebagai SQL, karena query pencatatan memakai prepared statement.

c) Ketiga, metode Session Hijacking juga tidak akan berjalan, karena cookie token yang diremember sekalipun disimpan untuk dicek IP dan User Agentnya. Sehingga apabila cookie tersebut dicuri sekalipun, karena IP dan User Agent yang berbeda login tidak dapat dilakukan juga.

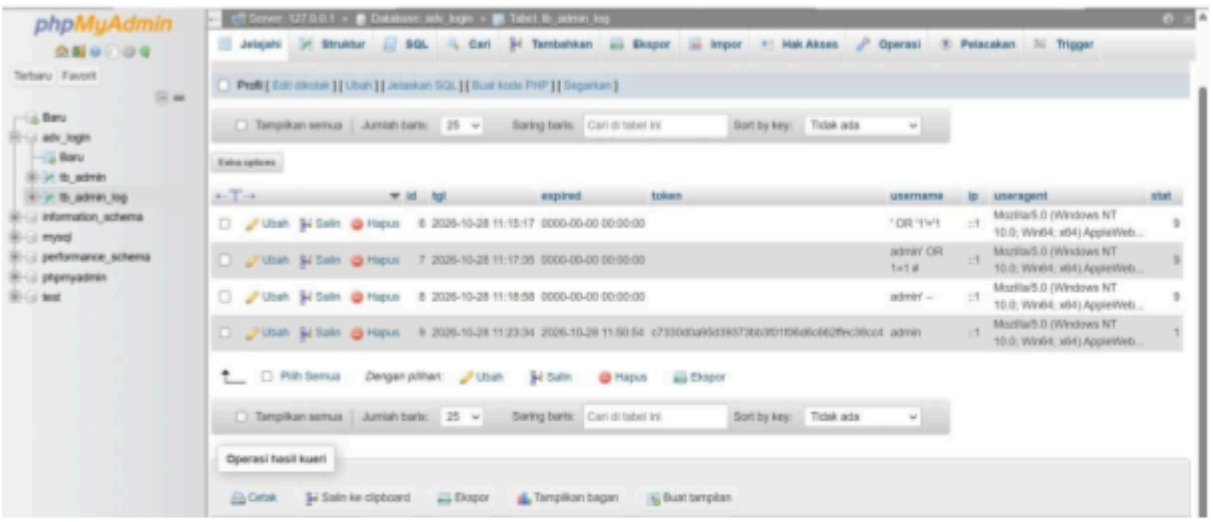
Berhasil login dan masuk ke home.php



(1) Mencoba Logout dari halaman home.php



Penjelasan:
fitur *logout* pada halaman index.php yang ditandai dengan munculnya pesan notifikasi "Success: Anda sudah logout dari sistem" setelah pengguna menekan tautan *Logout* di halaman utama. Saat proses ini dijalankan melalui file logout.php, sistem memanggil *method* $login->logout() yang bekerja menghentikan sesi pengguna secara menyeluruh baik di sisi klien maupun server. Di sisi klien (browser), sistem menghapus *cookie* adv_token menggunakan fungsi unset() dan setcookie() agar token tidak lagi tersimpan di perangkat pengguna. Sementara itu di sisi server (database), sistem memperbarui kolom expired pada tabel tb_admin_log menjadi waktu saat ini ($now) untuk token terkait, sehingga meskipun token tersebut sebelumnya sempat dicuri oleh pihak lain, token tersebut sudah tidak valid dan tidak dapat digunakan lagi untuk melakukan *Session Hijacking*. Setelah proses penghapusan dan invalidasi token selesai, fungsi create_alert() menyimpan pesan sukses ke dalam *session* dan mengalihkan pengguna kembali ke halaman form login (index.php), di mana akses langsung kembali ke halaman home.php tanpa autentikasi ulang akan otomatis ditolak oleh fungsi login_redir().



Penjelasan:
Tabel ini menunjukkan data yang dipakai server untuk memvalidasi setiap permintaan, yaitu token, IP, User-Agent, dan expired. Kolom expired yang berubah menjadi waktu logout membuktikan token dinonaktifkan di sisi server.

7. Kesimpulan

Hasil pengujian menunjukkan bahwa perlindungan terhadap brute force dan SQL injection pada modul ini bekerja sesuai klaim. Sedangkan perlindungan terhadap session hijacking baru efektif setelah kondisi validasi IP dan User-Agent diperketat dari || menjadi &&. Perlindungan ini tetap memiliki batas, karena User-Agent dapat dipalsukan dan penyerang pada jaringan yang sama dapat memiliki IP yang sama. Perlindungan yang lebih kuat dapat dicapai dengan menambahkan atribut cookie HttpOnly, Secure, dan SameSite serta menggunakan HTTPS.